

Newton Game Dynamics: Architecture and Design Principles

Introduction

Newton Game Dynamics is a rigid-body simulation library designed around a small set of physical and numerical principles rather than a collection of independent algorithms. Every major subsystem—from collision detection to articulated-body dynamics—follows the same design philosophy: preserve the physical model whenever practical, minimize numerical error, and expose interfaces that allow additional physics systems to participate naturally in the simulation.

Rather than viewing the engine as a collection of isolated features, it is more useful to think of it as a force-equilibrium solver. Every subsystem ultimately contributes to solving the same physical problem: computing a consistent set of accelerations and constraint forces for the current state of the world.

This document describes the principles behind that architecture.

Fundamental Design Principles

Conservation Before Approximation

Whenever possible, Newton attempts to preserve the governing equations of motion instead of replacing them with empirical corrections.

The objective is not to reproduce every aspect of reality perfectly—an impossible goal for any numerical simulation—but to remain as faithful as practical to the underlying physical model. When approximations become necessary, they are introduced explicitly and locally rather than becoming the foundation of the solver.

This philosophy influences every major subsystem of the engine.

The Scene as a Unified Dynamical System

Newton treats the entire simulation as a single coupled system.

Rigid bodies, articulated mechanisms, contacts, motors, particle systems, cloth, and any future force-based subsystem all participate in the same force equilibrium.

Conceptually, every body satisfies

$$Ma = F_{\text{user}} + F_{\text{velocity}} + J^T \lambda$$

where:

- **Fuser** represents externally applied forces supplied by the application.
- **Fvelocity** contains velocity-dependent forces such as gyroscopic and Coriolis effects.
- **J^Tλ** represents the internal constraint forces generated by contacts and joints.

The internal force vector is not a final result—it is a mutable force accumulator. Any subsystem may read or modify these forces while solving its own portion of the system, allowing specialized solvers to cooperate without requiring a monolithic global algorithm.

Temporal Coherence

Solving Fewer Problems Instead of Cheaper Problems

Newton is designed around temporal coherence.

Successive simulation frames are usually very similar. Instead of treating every frame as an independent collision-detection problem, Newton attempts to recognize when previously computed information remains valid.

For contact generation, the engine caches complete contact manifolds together with the relative motion of the interacting bodies.

A manifold is regenerated only when the accumulated relative displacement or relative motion exceeds configurable limits.

Because these tests involve only inexpensive scalar comparisons, many expensive narrow-phase collision computations can be skipped entirely.

The objective is not to make individual contact generation faster, but to reduce how often contact generation is required.

This allows Newton to invest more computation into producing higher-quality contact manifolds whenever regeneration becomes necessary.

Contact Manifolds

Whenever a contact manifold must be regenerated, Newton computes a complete manifold from the current geometry.

The engine does not rely on incrementally updating previous contact points as the primary representation of the current contact state.

Each regenerated manifold is therefore derived directly from the current geometric configuration.

This approach naturally scales with object size because the validity test depends on the relative motion of the interacting bodies rather than fixed world-space tolerances.

Constraint Solver

Acceleration-Level Formulation

The constraint solver operates primarily at the acceleration level.

Each constraint contributes equations describing the desired acceleration behavior of the constrained bodies.

These equations are assembled into the global system and solved simultaneously, producing physically consistent constraint forces for the entire scene.

The solver itself contains no knowledge of individual joint types.

Every constraint contributes only its Jacobian and its acceleration equations through a common interface.

Constraint Responsibilities

Each constraint implementation is responsible for describing its own physical behavior.

Every constraint provides two fundamental operations:

- construction of its Jacobian and acceleration equations;
- evaluation of the intermediate derivatives required during numerical integration.

This includes contacts, bilateral joints, motors, friction constraints, articulated models, and any future constraint type.

The numerical integrator never attempts to infer the physics of a constraint.

Instead, each constraint supplies the derivatives appropriate for its own physical model.

The integrator merely integrates those derivatives.

This separation keeps the numerical integration independent of the physical behavior being modeled.

Numerical Integration

Newton uses fourth-order Runge-Kutta integration for rigid-body dynamics.

The intermediate derivative evaluations are not generated by the integrator itself.

Instead, each constraint computes the intermediate derivatives appropriate for its own physical behavior and places them into the shared constraint descriptor.

Examples include:

- bilateral constraints using implicit derivative formulations;
- unilateral constraints using numerical derivative evolution;
- friction constraints evolving according to the current contact state;
- restitution handling supplying the appropriate intermediate acceleration changes.

RK4 simply integrates the derivatives supplied by the constraints.

As a result, discontinuities are handled individually by each constraint type rather than by the integrator itself.

Generalized Forces

The solution of the constraint system produces more than body accelerations.

It also produces generalized constraint-force vectors.

These vectors describe the forces required to satisfy every active constraint.

Because the solver naturally computes these quantities, they become available to other systems without requiring separate inverse-dynamics algorithms.

Articulated-Body Solver

Newton includes a direct articulated-body solver capable of handling closed kinematic loops together with contact constraints.

Rather than existing as a separate simulation pipeline, articulated models participate directly in the same global force equilibrium as every other constraint.

The articulated solver can compute generalized accelerations and generalized force vectors for the mechanism while remaining coupled to the surrounding rigid-body simulation.

The resulting constraint forces are injected back into the global force accumulator, allowing the reduced residual to propagate naturally to the remaining constraints.

Inverse Dynamics

Applications can access the articulated-body solver through the `ndIKSolver` interface.

After the contact graph has been generated—but before constraint resolution begins—the application may modify joint parameters provided the topology remains unchanged.

During this phase the application may:

- enable or disable joint degrees of freedom,
- modify motors,
- adjust limits,
- configure inverse-kinematics targets.

The solver computes the least-constrained acceleration together with the corresponding generalized constraint forces.

These generalized forces may then be projected onto the Jacobians of individual actuators to obtain the physical actuator forces required to realize the desired motion.

This allows industrial manipulators, robotic mechanisms, and other articulated systems to be controlled directly through physically meaningful actuator quantities.

Extensible Force-Based Architecture

One consequence of the unified force formulation is that new force-based simulation systems can participate naturally in the solver.

Particle systems, cloth solvers, fluid solvers, and other physically based models may read the accumulated internal forces, perform their own specialized computations, and inject their resulting force contributions back into the global equilibrium.

Because every subsystem communicates through forces and accelerations, each remains specialized while contributing to the same physical solution.

Numerical Robustness

Newton continuously monitors the conditioning of its constraint systems.

Debug builds provide diagnostics for positive semi-definite verification and matrix conditioning.

When necessary, regularization is applied locally to preserve numerical stability while minimizing its effect on the physical solution.

This allows pathological configurations to be detected explicitly rather than remaining hidden numerical failures.

Design Philosophy

Newton is built around a simple idea.

Every subsystem should preserve as much physical information as practical, contribute naturally to a common force-equilibrium formulation, and expose interfaces that allow additional physics systems to participate without changing the architecture.

The result is a simulation framework whose major components are not independent features but cooperating parts of the same mathematical model.